

Lab Sheet 08

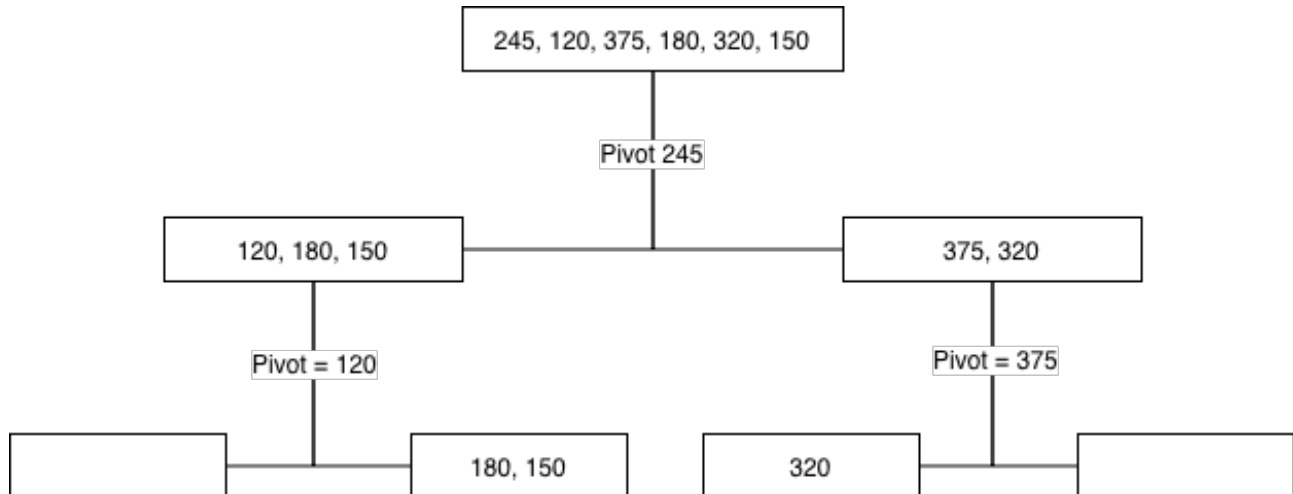
Registration No. : 2025/CS/079

Index No. : 25000799

A.

1. Pivot is used as a reference point to divide the given dataset into smaller, more manageable sub-problems. In each recursive step a pivot point is selected and the dataset is divided into smaller parts, until it cannot further be divided.

2.



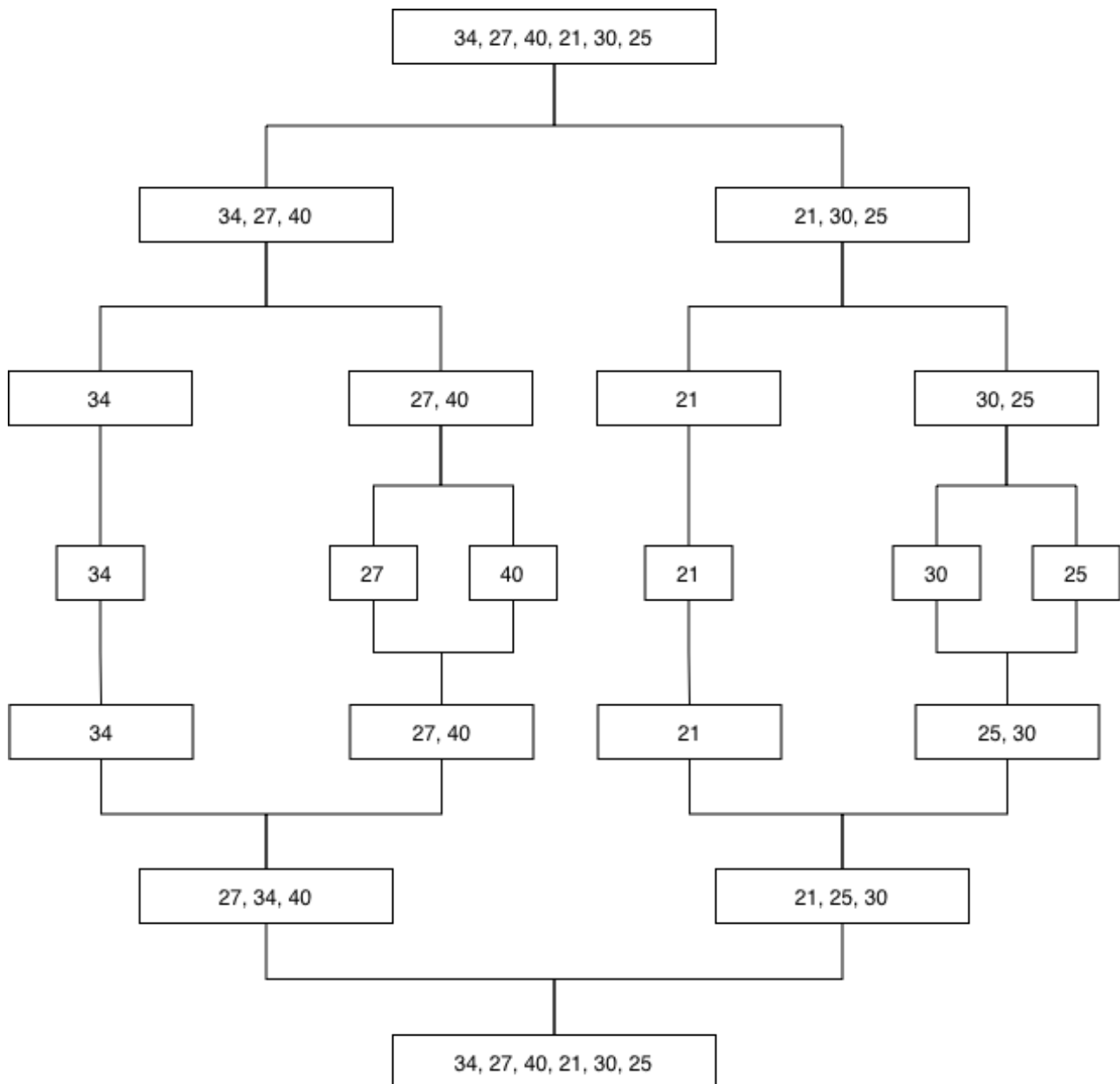
3. Worst case comparisons = $C(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$

$$= C(5000) = \frac{5000 * (5000 - 1)}{2}$$
$$= 12,497,500$$

4. Selecting a random pivot point.

B.

1. The list is divided into two parts from the middle, and keep dividing it until only a single element is left in the list. Then merge the relevant elements of the list while sorting them to be in the correct order, until all the lists are merged back together.



2. Time complexity $T(n) = n \log n$
 $T(1000) = 1000 * \log(1000)$
 $T(1000) = 9,966$
3. Since all the servers are working simultaneously, the problem can be solved faster than before. Also the efficiency of the sorting can be increased as smaller tasks can be handled easily.

C.**1.**

$$M1 = (A11 + A22) * (B11 + B22) = (3 + 4) * (5 + 3) = 56$$

$$M2 = (A21 + A22) * B11 = (1 + 4) * 5 = 25$$

$$M3 = A11 * (B12 - B22) = 3 * (1 - 3) = -6$$

$$M4 = A22 * (B21 - B11) = 4 * (2 - 5) = -12$$

$$M5 = (A11 + A12) * B22 = (3 + 2) * 3 = 15$$

$$M6 = (A21 - A11) * (B11 + B12) = (1 - 3) * (5 + 1) = -12$$

$$M7 = (A12 - A22) * (B21 + B22) = (2 - 4) * (2 + 3) = -10$$

$$C11 = M1 + M4 - M5 + M7 = 56 + (-12) - 15 + (-10) = 19$$

$$C12 = M3 + M5 = -6 + 15 = 9$$

$$C21 = M2 + M4 = 25 + (-12) = 13$$

$$C22 = M1 - M2 + M3 + M6 = 56 - 25 + (-6) + (-12) = 13$$

$$C = \begin{bmatrix} 19 & 9 \\ 13 & 13 \end{bmatrix}$$

- 2.** Normal matrix multiplication will require 8 multiplications. But in Strassen's Matrix multiplication only 7 multiplications are required. Therefore the n^3 time complexity can be reduced to $n^{\log 7}$. Therefore for larger matrices it will affect largely since that slight reduce in the power will affect the time complexity largely.

D.

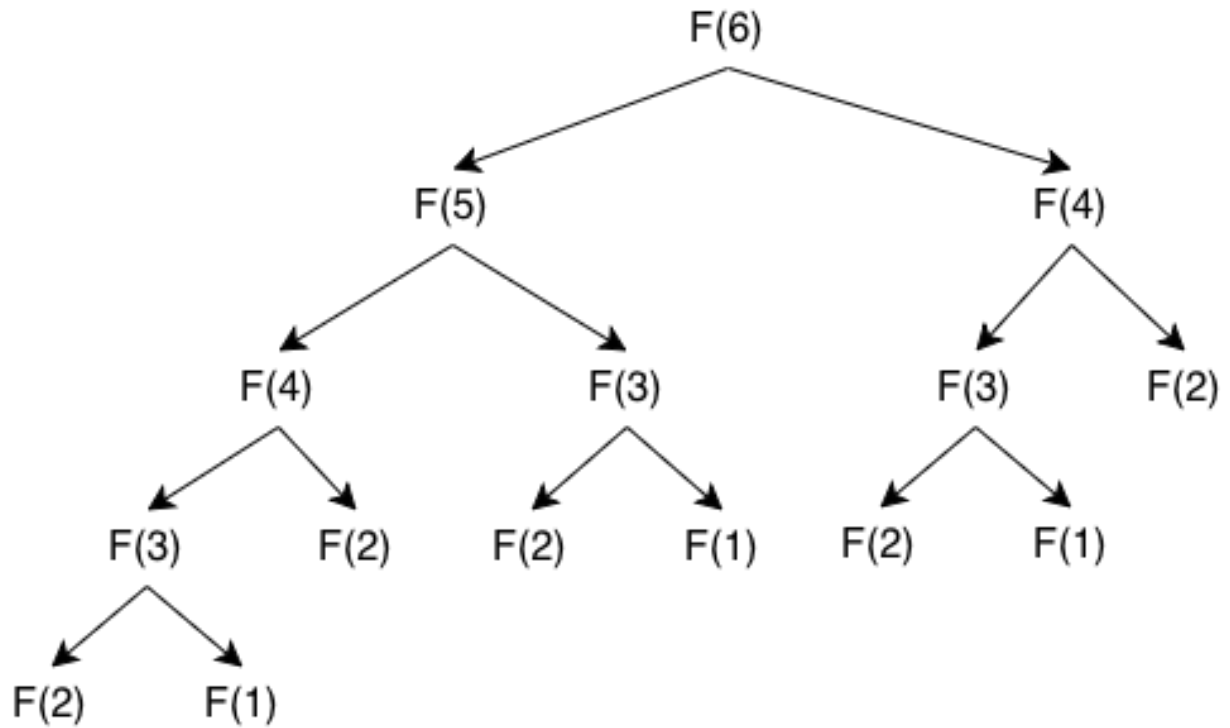
- 1.** When dividing a problem into subproblems using decomposition, there might be sub problems which require same calculation to solve them. Those sub problems with similar behaviour are called overlapping sub problems.

In normal recursion the function stores all the solutions for a subproblems in a call stack. To do so, the function might have to perform the same calculation over and over again. Dynamic programming approach prevents this by saving the calculated output of a subproblem in an array. Therefore dynamic programming can be more efficient than normal recursion.

2.

N	F(n-1) + F(n-2)	F(n)
1	Base case	1
2	Base case	2
3	2 + 1	3
4	3 + 2	5
5	5 + 3	8
6	8 + 5	13
7	13 + 8	21
8	21 + 13	34

3.



$F(4)$ is calculated twice.

$F(3)$ is calculated three times.

4. Naive recursion takes a time complexity of $O(2^n)$ while dynamic programming approach in tabulation method only takes $O(n)$. Since overlapping sub problems aren't calculated over and over again, the dynamic programming approach is much faster than normal recursion method.